

Beyond encryption at rest

Computing on encrypted data

Agenda

- Data location and security
- Homomorphic encryption
- Noise
- FHE scheme
- Code examples with demo
- Performance analysis
- Takeaways

Where your data lives

and how to secure it

Network

Disk

Memory

Where your data lives

and how to secure it

Network

- Encryption in transit

Disk

Memory

Where your data lives

and how to secure it

Network

- Encryption in transit

Disk

- Encryption at rest

Memory

Where your data lives

and how to secure it

Network

- Encryption in transit

Disk

- Encryption at rest
- But not with provider hosted keys

Memory

Memory is still an issue

Computing requires plaintext in RAM

- Meltdown
- Spectre
- Heartbleed
- Cold boot attacks
- Log4Shell

Where your data lives

and how to secure it

Network

- Encryption in transit

Disk

- Encryption at rest
- But not with provider hosted keys

Memory

Where your data lives

and how to secure it

Network

- Encryption in transit

Disk

- Encryption at rest
- But not with provider hosted keys

Memory

- Homomorphic Encryption

Homomorphic Encryption

An encryption scheme that mirror operations in plaintext

- $\text{enc}(M_1) + \text{enc}(M_2) = \text{enc}(M_1 + M_2)$
- $\text{enc}(M_1) \times \text{enc}(M_2) = \text{enc}(M_1 \times M_2)$

Homomorphic Encryption

An encryption scheme that mirrors sum and multiplication ops in plaintext

- $\text{enc}(M_1) + \text{enc}(M_2) = \text{enc}(M_1 + M_2)$
- $\text{enc}(M_1) \times \text{enc}(M_2) = \text{enc}(M_1 \times M_2)$

Textbook RSA is multiplicatively homomorphic

RSA encryption scheme

$$\text{enc}(M) = M^e \pmod{n}$$

$$\text{dec}(C) = C^d \pmod{n}$$

M cleartext message

e public exponent

n large number ($p \times q$)

C cyphertext

d private exponent

$$\phi(n) = (p-1)(q-1)$$

$$1 < e < \phi(n) \text{ and } e \text{ is coprime to } \phi(n)$$

$$d \cdot e \equiv 1 \pmod{\phi(n)}$$

RSA homomorphism example

$$p = 61, q = 53, n = 3233$$

$$e = 17 \quad (\text{coprime with } 3120)$$

$$\phi(n) = (p-1)(q-1) = 3120$$

$$d = 2753 \quad (17 \times d) \bmod 3120 = 1$$

RSA homomorphism example

$$\begin{array}{lll} p = 61, q = 53, n = 3233 & e = 17 & (\text{coprime with } 3120) \\ \phi(n) = (p-1)(q-1) = 3120 & d = 2753 & (17 \times d) \bmod 3120 = 1 \end{array}$$

$$M_1 = 65, M_2 = 12$$

RSA homomorphism example

$$p = 61, q = 53, n = 3233$$

$$e = 17$$

$$(\text{coprime with } 3120)$$

$$\phi(n) = (p-1)(q-1) = 3120$$

$$d = 2753$$

$$(17 \times d) \bmod 3120 = 1$$

$$M_1 = 65, M_2 = 12$$

$$\text{enc}(M_1) \times \text{enc}(M_2)$$

$$\text{enc}(M_1 \times M_2)$$

$$C_1 = M_1^e \bmod n = 65^{17} \bmod 3233 = 2790$$

$$C_2 = M_2^e \bmod n = 12^{17} \bmod 3233 = 1730$$

$$C_1 \times C_2 = (2790 \times 1730) \bmod 3233 = 3064$$

RSA homomorphism example

$$p = 61, q = 53, n = 3233$$

$$e = 17$$

$$(\text{coprime with } 3120)$$

$$\phi(n) = (p-1)(q-1) = 3120$$

$$d = 2753$$

$$(17 \times d) \bmod 3120 = 1$$

$$M_1 = 65, M_2 = 12$$

$$\text{enc}(M_1) \times \text{enc}(M_2)$$

$$=$$

$$\text{enc}(M_1 \times M_2)$$

$$C_1 = M_1^e \bmod n = 65^{17} \bmod 3233 = 2790$$

$$M_1 \times M_2 = 65 \times 12 = 780$$

$$C_2 = M_2^e \bmod n = 12^{17} \bmod 3233 = 1730$$

$$C = M_{12}^e \bmod n = 780^{17} \bmod 3233 = 3064$$

$$C_1 \times C_2 = (2790 \times 1730) \bmod 3233 = 3064$$

RSA homomorphism example

$$p = 61, q = 53, n = 3233$$

$$e = 17$$

$$(\text{coprime with } 3120)$$

$$\phi(n) = (p-1)(q-1) = 3120$$

$$d = 2753$$

$$(17 \times d) \bmod 3120 = 1$$

$$M_1 = 65, M_2 = 12$$

$$\text{enc}(M_1) \times \text{enc}(M_2)$$

=

$$\text{enc}(M_1 \times M_2)$$

$$C_1 = M_1^e \bmod n = 65^{17} \bmod 3233 = 2790$$

$$M_1 \times M_2 = 65 \times 12 = 780$$

$$C_2 = M_2^e \bmod n = 12^{17} \bmod 3233 = 1730$$

$$C = M_{12}^e \bmod n = 780^{17} \bmod 3233 = 3064$$

$$C_1 \times C_2 = (2790 \times 1730) \bmod 3233 = 3064$$

$$\text{enc}(M_1) + \text{enc}(M_2)$$

$$\text{enc}(M_1 + M_2)$$

$$C_1 + C_2 = (2790 + 1730) \bmod 3233 = 1287$$

RSA homomorphism example

$$p = 61, q = 53, n = 3233$$

$$e = 17$$

$$(\text{coprime with } 3120)$$

$$\phi(n) = (p-1)(q-1) = 3120$$

$$d = 2753$$

$$(17 \times d) \bmod 3120 = 1$$

$$M_1 = 65, M_2 = 12$$

$$\text{enc}(M_1) \times \text{enc}(M_2)$$

$$=$$

$$\text{enc}(M_1 \times M_2)$$

$$C_1 = M_1^e \bmod n = 65^{17} \bmod 3233 = 2790$$

$$M_1 \times M_2 = 65 \times 12 = 780$$

$$C_2 = M_2^e \bmod n = 12^{17} \bmod 3233 = 1730$$

$$C = M_{12}^e \bmod n = 780^{17} \bmod 3233 = 3064$$

$$C_1 \times C_2 = (2790 \times 1730) \bmod 3233 = 3064$$

$$\text{enc}(M_1) + \text{enc}(M_2)$$

$$\neq$$

$$\text{enc}(M_1 + M_2)$$

$$C_1 + C_2 = (2790 + 1730) \bmod 3233 = 1287$$

$$M_1 + M_2 = 65 + 12 = 77$$

$$C = M_{12}^e \bmod n = 77^{17} \bmod 3233 = 3123$$

Homomorphic Encryption Levels

$$\text{enc}(M_1) + \text{enc}(M_2) = \text{enc}(M_1 + M_2)$$

$$\text{enc}(M_1) \times \text{enc}(M_2) = \text{enc}(M_1 \times M_2)$$

- Partially Homomorphic Encryption: ∞ times, + XOR $\times$

Learning With Errors

Given many (a, b) pairs where $b = (a \cdot s + e) \bmod q$

- q a fixed number which sets the upper bound for a and b
- s a single fixed number for all the pairs
- a a random number in $[0, q-1]$
- e a small random number, close to 0, like $\{-1, 0, 1\}$

is very hard to find the value of s because of e (and $\bmod q$)

(this is a simplified version of LWE)

BFV Encryption Scheme

built on top of LWE (again, simplified)

Cyphertext is (c_0, c_1) where $c_1 = a$
 $c_0 = -a \cdot s + e + (q / t) \cdot m \pmod{q}$

q a fixed number defining the modulo all values wrap around

t a fixed number for the range of valid messages

s the secret key (the same for all the pairs)

a a random number $\in [0, q-1]$

e a small random number, close to 0, like $\{-2, -1, 0, 1, 2\}$

m the cleartext message $\in [0, t-1]$

Noise

$$c_1 = a, \quad c_0 = -a \cdot s + e + (q / t) \cdot m \pmod{q}$$



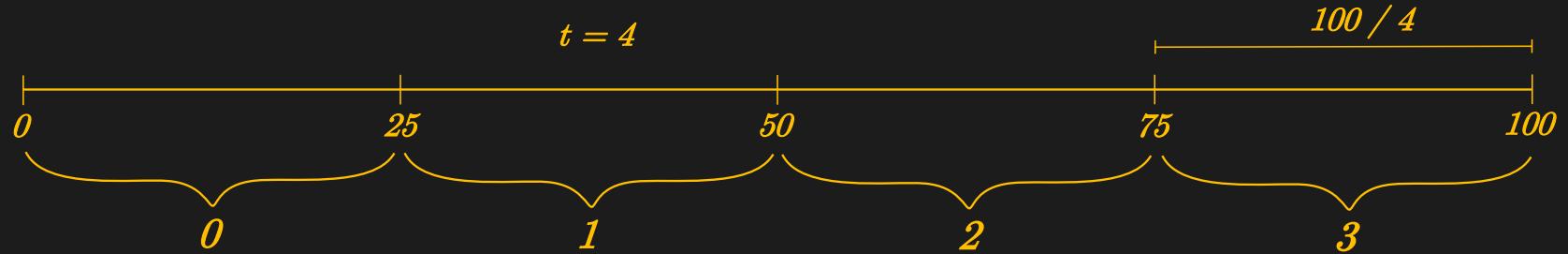
Noise

$$c_1 = a, \quad c_0 = -a \cdot s + e + (q / t) \cdot m \pmod{q}$$



Noise

$$c_1 = a, \quad c_0 = -a \cdot s + e + (q / t) \cdot m \pmod{q}$$



Noise

$$c_1 = a, \quad c_0 = -a \cdot s + e + (q / t) \cdot m \pmod{q}$$



Noise

$$c_1 = a, \quad c_0 = -a \cdot s + e + (q / t) \cdot m \pmod{q}$$



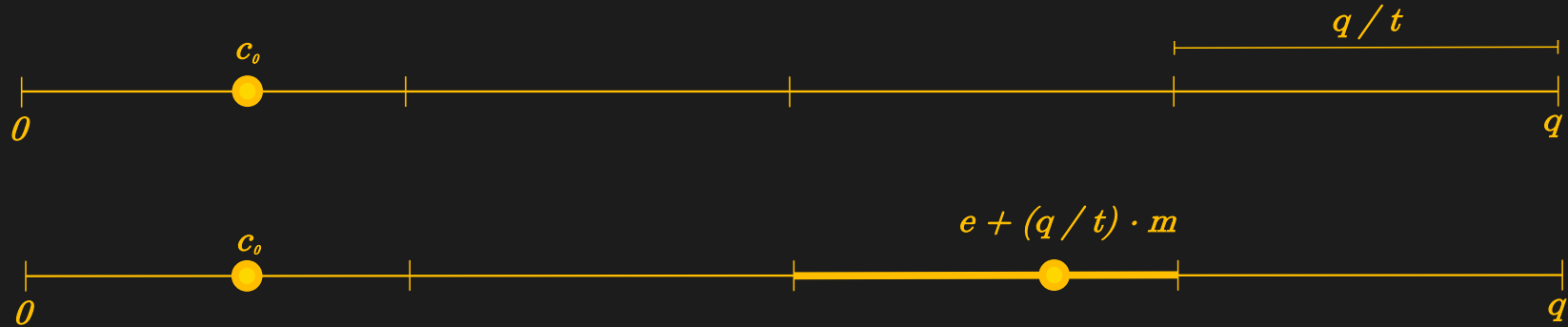
Noise

$$c_1 = a, \quad c_0 = -a \cdot s + e + (q / t) \cdot m \pmod{q}$$



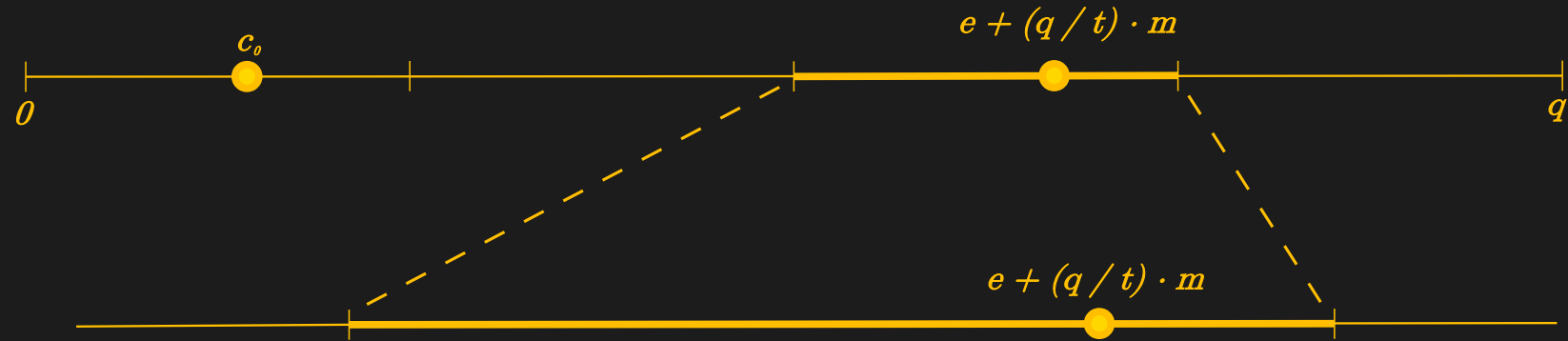
Noise

$$c_1 = a, \quad c_0 = -a \cdot s + e + (q / t) \cdot m \pmod{q}$$



Noise

$$c_1 = a, \quad c_0 = -a \cdot s + e + (q / t) \cdot m \pmod{q}$$



Noise

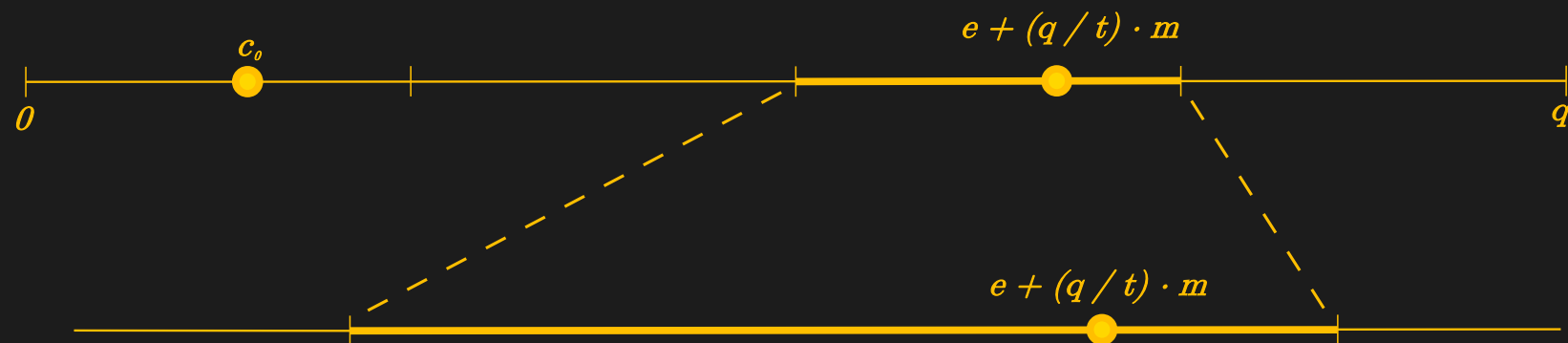
$$c_1 = a, \quad c_0 = -a \cdot s + e + (q/t) \cdot m \pmod{q}$$



after some operations

Noise

$$c_1 = a, \quad c_0 = -a \cdot s + e + (q/t) \cdot m \pmod{q}$$



after some operations



after some other operations



Noise

$$q = 20, t = 4$$



Noise

$$q = 20, t = 4$$



$$q = 20, t = 8$$



Noise

$$q = 20, t = 4$$



$$q = 20, t = 8$$



$$q = 20, t = 2$$



Homomorphic Encryption Levels

$$\text{enc}(M_1) + \text{enc}(M_2) = \text{enc}(M_1 + M_2)$$

$$\text{enc}(M_1) \times \text{enc}(M_2) = \text{enc}(M_1 \times M_2)$$

- Partially Homomorphic Encryption: ∞ times, + XOR $\times$
- Somewhat Homomorphic Encryption: ε times, + AND $\times$

Bootstrapping

Homomorphical Decryption of the cyphertext

In BFV we decrypt the cyphertext homomorphically using an encrypted s provided by whom created s

Bootstrapping

Homomorphical Decryption of the cyphertext

In BFV we decrypt the cyphertext homomorphically using an encrypted s provided by whom created s

$$m = \lfloor (t/q) \cdot [c_0 + c_1 \cdot s] \bmod q \rfloor \bmod t$$

Bootstrapping

Homomorphical Decryption of the cyphertext

In BFV we decrypt the cyphertext homomorphically using an encrypted s provided by whom created s

$$m = \lfloor (t/q) \cdot [c_0 + c_1 \cdot s] \bmod q \rfloor \bmod t$$

It's an expensive operation, so usually schemes are “leveled”

Homomorphic Encryption Levels

$$\text{enc}(M_1) + \text{enc}(M_2) = \text{enc}(M_1 + M_2)$$

$$\text{enc}(M_1) \times \text{enc}(M_2) = \text{enc}(M_1 \times M_2)$$

- Partially Homomorphic Encryption: ∞ times, + XOR $\times$
- Somewhat Homomorphic Encryption: ε times, + AND $\times$
- Fully Homomorphic Encryption: ∞ times, + AND $\times$

Comparisons and Branching

There is no “if” on encrypted data.

$a > b$ becomes a polynomial approximation.

The single most expensive thing to compute.

Branching

- CKKS: approximate $\text{sign}(x)$ with a polynomial, refined by Newton's method
- TFHE: bootstrap through a lookup table — the function lives inside the reset

Either way, “if” becomes a full homomorphic evaluation, not a branch

Branching example

This application receives sensitive encrypted data from the users and processes them, providing a cleartext output.

Coding example

```
// Encode weights as plaintext
weights := make([]float64, params.MaxSlots())
for i, w := range common.ModelWeights {
    weights[i] = w
}
ptWeights := hefloat.NewPlaintext(params, ctFeatures.Level())
if err := encoder.Encode(weights, ptWeights); err != nil {
    sendError(w, "Failed to encode weights", http.StatusInternalServerError)
    return
}

// Multiply encrypted features by weights (element-wise)
ctWeighted, err := evaluator.MulNew(ctFeatures, ptWeights)
if err != nil {
    sendError(w, "Failed to multiply", http.StatusInternalServerError)
    return
}
if err := evaluator.Rescale(ctWeighted, ctWeighted); err != nil {
    sendError(w, "Failed to rescale", http.StatusInternalServerError)
    return
}
fmt.Println("    ✓ Element-wise multiplication: Enc(features) × weights")
```

Let's see it running

Performance Analysis

- Addition: nearly free. Multiplication: costly.
- Logistic regression: <1ms plain, ~70ms encrypted.
- Overhead is shrinking — 10× every two years.

Choosing a Scheme

- CKKS: approximate reals, built for machine learning.
- BGV / BFV: exact integers, built for logic.
- TFHE: bit by bit, built for branching.

Scenarios

- Confidential AI: predictions with no data exposed
- Private credit scoring: live today, built by Zama
- Private genomics: GWAS across encrypted genomes

Takeaway

Encryption at rest isn't a safe option for some scenarios

Is your data worth FHE's extra cost?

Questions?

Download this presentation here:

